

Edge Addition Planarity Suite and Generalized Graph Library

John M. Boyer¹ and Wanda B. K. Boyer¹

¹ Independent Researcher, Canada Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

A *graph* is a mathematical structure comprising a set V of vertices and a set E of edges, with each edge e corresponding to a pair of vertices called the *endpoints* of e . A graph is *planar* if its vertices can be placed in distinct locations on a plane surface and its edges can be drawn on the plane without the edges intersecting, except at their common vertex endpoints. For a planar graph, a planarity algorithm typically outputs a combinatorial data structure that validation code can use to certify the planarity of the input graph. A planar graph drawing is typically produced by a separate algorithm. On the other hand, if an input graph is not planar, then a planarity algorithm typically outputs a minimal subgraph of the input graph that obstructs planarity. Validation code can use a minimal planarity-obstructing subgraph to certify the non-planarity of an input graph. Moreover, a minimal subgraph obstructing planarity can be used to help decide how to amend an input graph to *planarize* it.

Graphs are used to model a very wide array of real-world problems in which there are objects and relationships between the objects. In artificial intelligence, graphs are used to help with reasoning tasks, such as about the relationship pathways between persons of interest in law enforcement or between genes, tissues, diseases, and medications in bioinformatics research. In Physics, graphs and planarity are used to help compute material phase transitions, particle interactions, and electromagnetic duality. Similarly, in chemistry, graphs may be used to represent atoms and their valence bonds in molecules. Planarity testing can help determine feasible molecular arrangements because the molecular graphs for many compounds of interest must be planar. In electronics, a graph may be used to represent the electrical components and wiring in a circuit, such as transistors and etchings on a silicon wafer. The graph of a circuit must be planar or planarized to eliminate short circuits.

The open source software described in this paper provides a highly performant generalized graph library that also implements state-of-the-art algorithms for planarity-related problems. The generalized graph library and planarity-related algorithms are available to C/C++ and Python graph application developers.

Statement of need

The Edge Addition Planarity Suite is an open source software package that was originally developed to implement a new planarity algorithm in (J. M. Boyer & Myrvold, 2004). The Edge Addition Planarity Algorithm is currently regarded as one of two state-of-the-art planarity algorithms due to its conceptual simplicity as well as its speed of execution (contributors, 2026). The software package is available from a GitHub repository (J. M. Boyer, 2026) under a 3-clause BSD license.

To enable development of the Edge Addition Planarity algorithm, as well as the other planarity-related algorithms in (J. M. Boyer, 2006) and (J. M. Boyer, 2012), it was necessary to

41 develop an extensible, generalized graph library. This generalized graph library provides a
42 high-performance solution for the graph algorithm representation and processing needs of a
43 very wide array of graph applications.

44 In addition to enabling graph application development in C/C++, the popularity of Python as
45 a scientific application development language has prompted the need for making this
46 graph library and the algorithms it implements available to Python developers. It can be
47 accessed by a Github repository ([J. M. Boyer et al., 2026b](#)) and by simply using `pip install`
48 to get `planarity` from the Python package installer ([J. M. Boyer et al., 2026a](#)).

49 State of the field

50 Several large scientific software packages offer a graph planarity algorithm, which is a complex
51 graph algorithm suitable for benchmarking graph libraries. Mathematica and Wolfram Alpha
52 include planarity algorithms, but these packages are not meant for high performance graph
53 algorithms, so the

54 [Library for Efficient Data Types and Algorithms](#) (LEDA) is significantly faster because
55 it is an industrial library designed for computational efficiency. Yet, all planarity algorithms
56 implemented in LEDA were shown in ([J. M. Boyer et al., 2004](#)) to be several times slower than
57 the Edge Addition Planarity algorithm implementation. LEDA is also has the disadvantage of
58 being C++ only and has a proprietary license. The only graph library shown (in ([J. M. Boyer
59 et al., 2004](#))) to have similar performance to the Edge Addition Planarity Suite is the [Public
60 Implementation of a Graph Algorithm Library and Editor](#) (PIGALE). However, PIGALE is also
61 C++ only, has a GPLv2 license, and lacks advanced algorithms for homeomorphic subgraph
62 search. For example, torus embedding algorithms and torus obstruction isolation algorithms
63 are able to operate differently based on the result of a search for a subgraph homeomorphic to
64 $K_3, 3$ ([Myrvold & Kocay, 2011](#); [Myrvold & Woodcock, 2018](#)).

65 Software design

66 The generalized graph library in the Edge Addition Planarity Suite has a carefully curated API
67 and an object-oriented architecture. The base **Graph** class structure includes

- 68 ■ base graph, vertex, and edge structure definitions and getter/setter methods;
- 69 ■ graph, vertex and edge allocation, deallocation, and reset methods;
- 70 ■ methods for iterating through all vertices, edges, and edge adjacency lists;
- 71 ■ graph readers and writers for several formats; and
- 72 ■ a dynamic subclassing/extension mechanisms and virtual function table.

73 The base Graph is extendable with utility methods for exploring a graph, labelling vertices and
74 edges according to a depth-first search (DFS), and for managing connectivity and biconnectivity
75 information. The vertex and edge structures are extended to enable storage of the information
76 generated by the utility methods.

77 A Graph structure extended with the DFS-related utilities is further extendable to a **Planarity**
78 **Graph**, which also further extends the vertex and edge structures. The main additional method
79 provided by a Planarity Graph is `gp_Embed()`, which takes a Planarity Graph as input and
80 outputs either a combinatorial planar embedding or a minimal planarity-obstructing subgraph.

81 Given a Planarity Graph, a number of subclass extensions are available for planarity-related
82 problems. One extension solves outerplanarity. Another implements the planar graph drawing
83 algorithm in ([J. M. Boyer, 2006](#)). Three other extensions provide Graph subclasses that solve
84 the homeomorphic subgraph search algorithms appearing in ([J. M. Boyer, 2012](#)). All of these
85 extensions further extend the vertex and edge structures and overload the `gp_Embed()` function.

86 In addition to being available to C and C++ developers, the APIs of the Edge Addition

Planarity Suite and Generalized Graph Library are also made available via the planarity Python package (J. M. Boyer et al., 2026b). Cython is used to preserve high performance while also broadening availability to the Python developer community.

A final aspect of this open source software project is our emphasis on software reliability. We perform validation code on `gp_Embed()` and its five overloads on billions of randomly generated graphs up to 10 million vertices and 30 million edges. The GitHub actions that run on every pull request include sanitizer tests on all graphs on 8 vertices. Finally, the Edge Addition Planarity Suite Testing repository (W. B. K. Boyer & Boyer, 2026) provides our multithreaded Python code for validating `gp_Embed()` and its five overloads on the more than one billion graphs having 11 or fewer vertices, as generated by the geng tool in Nauty and Traces (McKay & Adolfo, 2025).

Research impact statement

The journal publication of the Edge Addition Planarity algorithm, (J. M. Boyer & Myrvold, 2004), currently has over 360 scholarly citations according to Google Scholar. The algorithm has been implemented in several large open source projects, including Boost, Magma, nauty and Traces (McKay & Adolfo, 2025), and the Open Graph Drawing Framework (web archive). The source code from the Edge Addition Planarity Suite repository has been directly included in several large open source projects, including SageMath, Digraphs, and over 80 Linux platforms releases including for Debian, Devuan, Kali, openSUSE, Raspbian, Ubuntu, Alpine, ALT, Fedora, FreeBSD, Gentoo, Manjaro, MSYS2, and Parabola, according to repology/edge-addition-planarity-suite (web archive) and repology/planarity (web archive).

The Python package named planarity was originally developed in 2013 by Aric Hagberg as a way to get the planarity testing and drawing algorithms from (J. M. Boyer & Myrvold, 2004) and (J. M. Boyer, 2006) to work with graphs from NetworkX (Hagberg et al., 2008). Recently, Hagberg transferred the planarity GitHub repository [PlanarityDevelopers:PlanarityGitHub] to the GitHub Graph Algorithms Organization and the planarity Python package (J. M. Boyer et al., 2026a) to the PyPI Graph Algorithms Organization, which are both maintained by the authors. On August 1, 2026, the Top PyPI Packages website indicated that the planarity Python package had over 700,000 downloads for the preceding month and was ranked in the top 1% of PyPI package downloads (ranked 5262 out of 864,036 projects). The planarity Python package has now been included as a standard Python package in several versions of Linux including Debian, Devuan, Kali, Raspbian, and Ubuntu, according to repology/python:planarity (web archive).

AI usage disclosure

No generative AI tools were used in the development of this software, the writing of this manuscript, nor the preparation of supporting materials.

References

- Boyer, J. M. (2006). A new method for efficiently generating planar graph visibility representations. In P. Eades & P. Healy (Eds.), *Proceedings of the 13th international symposium on graph drawing 2005* (Vol. 3843, pp. 508–511). Springer-Verlag. https://doi.org/10.1007/11618058_47
- Boyer, J. M. (2012). Subgraph homeomorphism via the edge addition planarity algorithm. *Journal of Graph Algorithms and Applications*, 16(2), 381–410. <https://doi.org/10.7155/jgaa.00268>

- 131 Boyer, J. M. (2026). Edge addition planarity suite, version 5.0.0.0. In *GitHub repository*.
132 GitHub. <https://github.com/graph-algorithms/edge-addition-planarity-suite>
- 133 Boyer, J. M., Boyer, W. B. K., & Hagberg, A. A. (2026a). Planarity: Python package, version
134 1.0.0. In *Python Package Installer*. PyPI. <https://pypi.org/project/planarity/>
- 135 Boyer, J. M., Boyer, W. B. K., & Hagberg, A. A. (2026b). Planarity: Python source code,
136 version 1.0.0. In *GitHub repository*. GitHub. [https://github.com/graph-algorithms/](https://github.com/graph-algorithms/planarity)
137 [planarity](https://github.com/graph-algorithms/planarity)
- 138 Boyer, J. M., Cortese, P. F., Patrignani, M., & Di Battista, G. (2004). Stop minding your
139 P's and Q's: Implementing a fast and simple DFS-based planarity testing and embedding
140 algorithm. In G. Liotta (Ed.), *Proceedings of the 11th international symposium on graph*
141 *drawing 2003* (Vol. 2912, pp. 25–36). Springer-Verlag. [https://doi.org/10.1007/978-3-](https://doi.org/10.1007/978-3-540-24595-7_3)
142 [540-24595-7_3](https://doi.org/10.1007/978-3-540-24595-7_3)
- 143 Boyer, J. M., & Myrvold, W. J. (2004). On the cutting edge: Simplified $O(n)$ planarity
144 by edge addition. *Journal of Graph Algorithms and Applications*, 8(3), 241–273. <https://doi.org/10.7155/jgaa.00091>
145 <https://doi.org/10.7155/jgaa.00091>
- 146 Boyer, W. B. K., & Boyer, J. M. (2026). Edge addition planarity suite testing. In *GitHub*
147 *repository*. GitHub. [https://github.com/graph-algorithms/edge-addition-planarity-suite-](https://github.com/graph-algorithms/edge-addition-planarity-suite-testing)
148 [testing](https://github.com/graph-algorithms/edge-addition-planarity-suite-testing)
- 149 contributors, W. (2026). Planarity testing. In *Wikipedia, the free encyclopedia*. Wikipedia.
150 https://en.wikipedia.org/w/index.php?title=Planarity_testing&oldid=1336627782
- 151 Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics,
152 and function using NetworkX. *Python in Science Conference*. [https://doi.org/10.25080/](https://doi.org/10.25080/TCWV9851)
153 [TCWV9851](https://doi.org/10.25080/TCWV9851)
- 154 McKay, B. D., & Adolfo, P. (2025). *Nauty and traces user's guide (version 2.9.0)*. <https://pallini.di.uniroma1.it/nug29.pdf>
155 <https://pallini.di.uniroma1.it/nug29.pdf>
- 156 Myrvold, W., & Kocay, W. (2011). Errors in graph embedding algorithms. *Journal of Computer*
157 *and System Sciences*, 77(2), 430–438. <https://doi.org/10.1016/j.jcss.2010.06.002>
- 158 Myrvold, W., & Woodcock, J. (2018). A large set of torus obstructions and how they were
159 discovered. *The Electronic Journal of Combinatorics*, 25(1), 1–17. [https://doi.org/10.](https://doi.org/10.37236/3797)
160 [37236/3797](https://doi.org/10.37236/3797)